

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ имени
И.С.ТУРГЕНЕВА»**

**ИНСТИТУТ ПРИБОРОСТРОЕНИЯ, АВТОМАТИЗАЦИИ И
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

Кафедра информационных систем и цифровых технологий

Захарова О.В.

**Методические указания к лабораторным работам
по дисциплине «Разработка профессиональных
приложений»**

для направлений подготовки: 09.03.01, 09.03.04

Орел 2022

Содержание

Лабораторная работа № 1	
Модульное программирование. Поточковый ввод/вывод	3
Лабораторная работа № 2	
Разработка класса. Работа с массивом объектов	15
Лабораторная работа № 3	
Работа со строками (класс string)	24
Лабораторная работа № 4	
Перегрузка операций и преобразование типов	30
Лабораторная работа № 5	
Наследование	34
Лабораторная работа № 6	
Работа со связным списком	36
Лабораторная работа № 7	
Работа со стандартной библиотекой шаблонов STL	37

В методических указаниях к лабораторным работам задания представлены по вариантам. Номер варианта задания выдает преподаватель каждому студенту.

Лабораторная работа № 1

Модульное программирование. Поточковый ввод/вывод

Вариант № 1

В программе разработать структуру «Сотрудник» с полями: табельный номер (целое число), ФИО (строка), адрес (строка), телефон (строка), пол (перечисление), дата рождения (структура «Дата»), должность (строка), оклад (вещественное число), наличие высшего образования (логический тип).

Вводимую информацию о сотрудниках хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом сотруднике (реализовать возможность вводить столько данных о сотруднике, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о сотруднике (перегруженные функции для удаления: по табельному номеру; по ФИО; по табельному номеру и ФИО);
- 3) отображения информации о сотрудниках фирмы в виде таблицы;
- 4) поиска информации о сотрудниках указанного года рождения;
- 5) определения количества сотрудников женского пола с высшим образованием;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Сотрудник», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 2

В программе разработать структуру «Учитель» с полями: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), адрес (строка), телефон (строка), преподаваемый предмет (строка), стаж работы (целое число), наличие ученой степени (логический тип).

Вводимую информацию об учителях хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом учителе (реализовать возможность вводить столько данных об учителе, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации об учителе (перегруженные функции для удаления: по табельному номеру; по Фамилии; по табельному номеру и Фамилии);
- 3) отображения информации об учителях в виде таблицы;
- 4) поиска информации об учителях указанного года рождения;
- 5) определения количества учителей мужского пола с ученой степенью;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Учитель», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 3

В программе разработать структуру «Ученик» с полями: код (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), класс (строка), наличие троек в четверти (логический тип).

Вводимую информацию об учениках хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом ученике (реализовать возможность вводить столько данных об ученике, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации об ученике (перегруженные функции для удаления: по коду; по Фамилии и Имени; по коду и Фамилии);

3) отображения информации об учениках в виде таблицы;

4) поиска информации об учениках указанного года рождения;

5) определения количества учеников женского пола, закончивших четверть без троек;

6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;

7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Ученик», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 4

В программе разработать структуру «Книга» с полями: ISBN (строка), название (строка), автор (структура «Автор»), издательство (строка), год выпуска (целое число), количество страниц (целое число), жанр (перечисление), наличие твердого переплета (логический тип).

Вводимую информацию о книгах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новой книге (реализовать возможность вводить столько данных о книге, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о книге (перегруженные функции для удаления: по ISBN; по году выпуска; по ISBN и году выпуска);

- 3) отображения информации о книгах в виде таблицы;
- 4) поиска информации о книгах указанного автора (по фамилии);
- 5) определения количества книг указанного жанра в твердом переплете;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Книга», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 5

В программе разработать структуру «Журнал» с полями: код (целое число), название (строка), дата выпуска (структура «Дата»), издательство (строка), количество страниц (целое число), тип (перечисление), периодичность (логический тип).

Вводимую информацию о журналах хранить в динамическом массиве структур (не использовать STL), объявленном в функции `main` (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом журнале (реализовать возможность вводить столько данных о журнале, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о журнале (перегруженные функции для удаления: по коду; по названию; по коду и названию);
- 3) отображения информации о журналах в виде таблицы;
- 4) поиска информации о журналах указанного года выпуска;
- 5) определения количества периодических журналов указанного типа;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Журнал», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 6

В программе разработать структуру «Товар» с полями: код (целое число), название (строка), дата выпуска (структура «Дата»), отдел (перечисление), стоимость (вещественное число), описание (строка), уцененный товар (логический тип).

Вводимую информацию о товарах хранить в динамическом массиве структур (не использовать STL), объявленном в функции `main` (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом товаре (реализовать возможность вводить столько данных о товаре, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о товаре (перегруженные функции для удаления: по коду; по названию; по коду и названию);

3) отображения информации о товарах в виде таблицы;

4) поиска информации о товарах указанного года выпуска;

5) определения количества уцененных товаров из указанного отдела;

6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;

7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Товар», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 7

В программе разработать структуру «Продавец» с полями: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), адрес (строка), телефон (строка), пол (перечисление), дата рождения (структура «Дата»), оклад (вещественное число), наличие высшего образования (логический тип).

Вводимую информацию о продавцах хранить в динамическом массиве структур (не использовать STL), объявленном в функции `main` (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом продавце (реализовать возможность вводить столько данных о продавце, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о продавце (перегруженные функции для удаления: по табельному номеру; по Фамилии и Имени; по табельному номер и Фамилии);

3) отображения информации о продавцах в виде таблицы;

4) поиска информации о продавцах указанного года рождения;

5) определения количества продавцов женского пола с высшим образованием;

6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;

7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Продавец», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 8

В программе разработать структуру «Договор» с полями: номер договора (целое число), ФИО клиента (строка), дата заключения договора (структура «Дата»), адрес клиента (строка), телефон клиента (строка), оказанная услуга (перечисление), стоимость услуги (вещественный тип), предоставлена скидка (логический тип).

Вводимую информацию о заключенных договорах на услуги хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом договоре (реализовать возможность вводить столько данных о договоре, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о договоре (перегруженные функции для удаления: по номеру договора; по ФИО клиента; по номеру договора и ФИО клиента);
- 3) отображения информации о договорах в виде таблицы;
- 4) поиска информации о договорах, заключенных в указанном году;
- 5) определения количества договоров, заключенных на указанную услугу, по которым предоставлена скидка;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Договор», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 9

В программе разработать структуру «Лекарство» с полями: код (целое число), название (строка), дата выпуска (структура «Дата»), стоимость (вещественное число), отдел (перечисление), наличие рецепта (логический тип), описание (строка).

Вводимую информацию о лекарствах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом лекарстве (реализовать возможность вводить столько данных о лекарстве, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о лекарстве (перегруженные функции для удаления: по коду; по названию; по коду и названию);
- 3) отображения информации о лекарствах в виде таблицы;
- 4) поиска информации о лекарствах указанного года выпуска;
- 5) определения количества лекарств из указанного отдела и отпускающихся по рецепту;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Лекарство», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 10

В программе разработать структуру «Фармацевт» с полями: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), адрес (строка), телефон (строка), пол (перечисление), дата рождения (структура «Дата»), оклад (вещественное число), наличие медицинского образования (логический тип).

Вводимую информацию о фармацевтах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом фармацевте (реализовать возможность вводить столько данных о фармацевте, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о фармацевте (перегруженные функции для удаления: по табельному номеру; по Фамилии и Имени; по табельному номер и Фамилии);

3) отображения информации о фармацевтах в виде таблицы;

4) поиска информации о фармацевтах указанного года рождения;

5) определения количества фармацевтов мужского пола с медицинским образованием;

6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;

7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Фармацевт», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 11

В программе разработать структуру «Преподаватель» с полями: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), адрес (строка), телефон (строка), преподаваемый предмет (строка), стаж работы (целое число), наличие ученой степени (логический тип).

Вводимую информацию о преподавателях хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом преподавателе (реализовать возможность вводить столько данных о преподавателе, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о преподавателе (перегруженные функции для удаления: по табельному номеру; по Фамилии; по табельному номеру и Фамилии);

- 3) отображения информации о преподавателях в виде таблицы;
- 4) поиска информации о преподавателях указанного года рождения;
- 5) определения количества преподавателей мужского пола с ученой степенью;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Преподаватель», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 12

В программе разработать структуру «Студент» с полями: номер зачетки (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), специальность (строка), наличие троек (логический тип).

Вводимую информацию о студентах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом студенте (реализовать возможность вводить столько данных о студенте, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о студенте (перегруженные функции для удаления: по номеру зачетки; по Фамилии и Имени; по номеру зачетки и Фамилии);
- 3) отображения информации о студентах в виде таблицы;
- 4) поиска информации о студентах указанного года рождения;
- 5) определения количества студентов женского пола, обучающихся без троек;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Студент», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 13

В программе разработать структуру «Школа» с полями: код (целое число), название (строка), город (строка), количество учеников (целое число), адрес (строка), телефон (строка), директор (структура «Директор»), тип (перечисление), государственная (логический тип).

Вводимую информацию о школах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новой школе (реализовать возможность вводить столько данных о школе, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о школе (перегруженные функции для удаления: по коду; по названию; по коду и названию);
- 3) отображения информации о школах в виде таблицы;
- 4) поиска информации о школах с указанным директором (по фамилии);
- 5) определения количества государственных школ указанного типа;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Школа», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 14

В программе разработать структуру «Университет» с полями: код (целое число), название (строка), город (перечисление), дата основания (структура «Дата»), ФИО ректора (строка), адрес (строка), телефон (строка), государственный (логический тип).

Вводимую информацию об университетах хранить в динамическом массиве структур (не использовать STL), объявленном в функции `main` (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом университете (реализовать возможность вводить столько данных об университетах, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации об университете (перегруженные функции для удаления: по коду; по названию; по коду и названию);
- 3) отображения информации об университетах в виде таблицы;
- 4) поиска информации об университетах, основанных в указанном году;
- 5) определения количества государственных университетов, расположенных в указанном городе;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Университет», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 15

В программе разработать структуру «Микроконтроллер» с полями: артикул (целое число), название (строка), фирма-производитель (структура «Фирма-производитель»), разрядность (целое число), тактовая частота (вещественное число), архитектура (перечисление), стоимость (вещественное число), уцененный товар (логический тип).

Вводимую информацию о микроконтроллерах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом микроконтроллере (реализовать возможность вводить столько данных о микроконтроллере, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о микроконтроллере (перегруженные функции для удаления: по артикулу; по названию; по артикулу и названию);
- 3) отображения информации о микроконтроллерах в виде таблицы;
- 4) поиска информации о микроконтроллерах указанной фирмы (по названию);
- 5) определения количества уцененных микроконтроллеров указанной архитектуры;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Микроконтроллер», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 16

В программе разработать структуру «Ученик» с полями: код (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), класс (строка), наличие троек в четверти (логический тип).

Вводимую информацию об учениках хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом ученике (реализовать возможность вводить столько данных об ученике, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации об ученике (перегруженные функции для удаления: по коду; по классу; по коду и классу);
- 3) отображения информации об учениках в виде таблицы;
- 4) поиска информации об учениках указанного года рождения;
- 5) определения количества учеников мужского пола, закончивших четверть без троек;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Ученик», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 17

В программе разработать структуру «Процессор» с полями: артикул (целое число), название (строка), фирма-производитель (структура «Фирма-производитель»), разрядность (целое число), тактовая частота (вещественное число), архитектура (перечисление), стоимость (вещественное число), уцененный товар (логический тип).

Вводимую информацию о процессорах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом процессоре (реализовать возможность вводить столько данных о процессоре, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о процессоре (перегруженные функции для удаления: по артикулу; по названию; по артикулу и названию);

3) отображения информации о процессорах в виде таблицы;

4) поиска информации о процессорах указанной фирмы (по названию);

5) определения количества уцененных процессоров указанной архитектуры;

6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;

7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции main разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию main), вспомогательный модуль (содержит структуру «Процессор», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 18

В программе разработать структуру «Студент» с полями: номер зачетки (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), специальность (строка), наличие троек (логический тип).

Вводимую информацию о студентах хранить в динамическом массиве структур (не использовать STL), объявленном в функции main (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

1) добавления информации о новом студенте (реализовать возможность вводить столько данных о студенте, сколько необходимо пользователю при помощи параметров по умолчанию);

2) удаления информации о студенте (перегруженные функции для удаления: по номеру зачетки; по Фамилии; по номеру зачетки и Фамилии);

3) отображения информации о студентах в виде таблицы;

- 4) поиска информации о студентах указанного года рождения;
- 5) определения количества студентов мужского пола, обучающихся без троек;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Студент», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 19

В программе разработать структуру «Лекарство» с полями: код (целое число), название (строка), дата выпуска (структура «Дата»), стоимость (вещественное число), отдел (перечисление), наличие рецепта (логический тип), описание (строка).

Вводимую информацию о лекарствах хранить в динамическом массиве структур (не использовать STL), объявленном в функции `main` (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом лекарстве (реализовать возможность вводить столько данных о лекарстве, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации о лекарстве (перегруженные функции для удаления: по коду; по названию и отделу; по коду и названию);
- 3) отображения информации о лекарствах в виде таблицы;
- 4) поиска информации о лекарствах указанного года выпуска;
- 5) определения количества лекарств из указанного отдела и отпускающихся без рецепта;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Лекарство», подпрограммы добавления, удаления, отображения и поиска информации).

Вариант № 20

В программе разработать структуру «Университет» с полями: код (целое число), название (строка), город (перечисление), дата основания (структура «Дата»), ФИО ректора (строка), адрес (строка), телефон (строка), государственный (логический тип).

Вводимую информацию об университетах хранить в динамическом массиве структур (не использовать STL), объявленном в функции `main` (главный модуль).

Разработать подпрограммы (вспомогательный модуль) для:

- 1) добавления информации о новом университете (реализовать возможность вводить столько данных об университетах, сколько необходимо пользователю при помощи параметров по умолчанию);
- 2) удаления информации об университете (перегруженные функции для удаления: по коду; по названию и городу; по коду и названию);
- 3) отображения информации об университетах в виде таблицы;
- 4) поиска информации об университетах, основанных в указанном году;
- 5) определения количества государственных университетов, расположенных в указанном городе;
- 6) сохранения данных из массива структур в файл (в бинарном режиме) по указанному пути;
- 7) чтения данных из файла (в бинарном режиме) по указанному пути в массив структур.

В главной функции `main` разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

В программе использовать потоковый ввод-вывод. В программе разработать встраиваемую функцию.

Структура программы: главный модуль (содержит функцию `main`), вспомогательный модуль (содержит структуру «Университет», подпрограммы добавления, удаления, отображения и поиска информации).

Лабораторная работа № 2

Разработка класса. Работа с массивом объектов

Вариант № 1

В программе разработать класс «Сотрудник» со свойствами: табельный номер (целое число), ФИО (строка), адрес (строка), телефон (строка), пол (перечисление), дата рождения (структура «Дата»), должность (строка), оклад (вещественное число), наличие высшего образования (логический тип), название фирмы (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о сотрудниках хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом сотруднике. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о сотруднике (по табельному номеру; по ФИО; по табельному номеру и ФИО);
- 3) отображения информации о сотрудниках фирмы в виде таблицы;
- 4) поиска информации о сотрудниках указанного года рождения;
- 5) определения количества сотрудников женского пола с высшим образованием;
- 6) изменения названия фирмы, в которой работают сотрудники. Разработать статический метод, позволяющий изменять название фирмы.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 2

В программе разработать класс «Учитель» со свойствами: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), адрес (строка), телефон (строка), преподаваемый предмет (строка), стаж работы (целое число), наличие ученой степени (логический тип), название школы (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию об учителях хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом учителе. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации об учителе (по табельному номеру; по Фамилии; по табельному номеру и Фамилии);
- 3) отображения информации об учителях в виде таблицы;
- 4) поиска информации об учителях указанного года рождения;
- 5) определения количества учителей мужского пола с ученой степенью;
- 6) изменения названия школы, в которой работают учителя. Разработать статический метод, позволяющий изменять название школы.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 3

В программе разработать класс «Ученик» со свойствами: код (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), класс (строка), наличие троек в

четверти (логический тип), название школы (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию об учениках хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом ученике. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации об ученике (по коду; по Фамилии и Имени; по коду и Фамилии);
- 3) отображения информации об учениках в виде таблицы;
- 4) поиска информации об учениках указанного года рождения;
- 5) определения количества учеников женского пола, закончивших четверть без троек;
- 6) изменения названия школы, в которой учатся ученики. Разработать статический метод, позволяющий изменять название школы.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 4

В программе разработать класс «Книга» со свойствами: ISBN (строка), название (строка), автор (структура «Автор»), издательство (строка), год выпуска (целое число), количество страниц (целое число), жанр (перечисление), наличие твердого переплета (логический тип), название библиотеки (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о книгах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новой книге. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о книге (по ISBN; по году выпуска; по ISBN и году выпуска);
- 3) отображения информации о книгах в виде таблицы;
- 4) поиска информации о книгах указанного автора (по фамилии);
- 5) определения количества книг указанного жанра в твердом переплете;
- 6) изменения названия библиотеки, в которой хранятся книги. Разработать статический метод, позволяющий изменять название библиотеки.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 5

В программе разработать класс «Журнал» со свойствами: код (целое число), название (строка), дата выпуска (структура «Дата»), количество страниц (целое число), тип (перечисление), периодичность (логический тип), название издательства (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о журналах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом журнале. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о журнале (по коду; по названию; по коду и названию);

- 3) отображения информации о журналах в виде таблицы;
- 4) поиска информации о журналах указанного года выпуска;
- 5) определения количества периодических журналов указанного типа;
- 6) изменения названия издательства, в котором издаются журналы. Разработать статический метод, позволяющий изменять название издательства.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 6

В программе разработать класс «Товар» со свойствами: код (целое число), название (строка), дата выпуска (структура «Дата»), отдел (перечисление), стоимость (вещественное число), описание (строка), уцененный товар (логический тип), название магазина (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о товарах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом товаре. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о товаре (по коду; по названию; по коду и названию);
- 3) отображения информации о товарах в виде таблицы;
- 4) поиска информации о товарах указанного года выпуска;
- 5) определения количества уцененных товаров из указанного отдела;
- 6) изменения названия магазина, в котором продаются товары. Разработать статический метод, позволяющий изменять название магазина.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 7

В программе разработать класс «Продавец» со свойствами: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), адрес (строка), телефон (строка), пол (перечисление), дата рождения (структура «Дата»), оклад (вещественное число), наличие высшего образования (логический тип), название магазина (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о продавцах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом продавце. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о продавце (по табельному номеру; по Фамилии и Имени; по табельному номеру и Фамилии);
- 3) отображения информации о продавцах в виде таблицы;
- 4) поиска информации о продавцах указанного года рождения;
- 5) определения количества продавцов женского пола с высшим образованием;
- 6) изменения названия магазина, в котором работают продавцы. Разработать статический метод, позволяющий изменять название магазина.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 8

В программе разработать класс «Договор» со свойствами: номер договора (целое число), ФИО клиента (строка), дата заключения договора (структура «Дата»), адрес клиента (строка), телефон клиента (строка), оказанная услуга (перечисление), стоимость услуги (вещественный тип), предоставлена скидка (логический тип), название фирмы (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о договорах на услуги хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом договоре. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о договоре (по номеру договора; по ФИО клиента; по номеру договора и ФИО клиента);
- 3) отображения информации о договорах в виде таблицы;
- 4) поиска информации о договорах, заключенных в указанном году;
- 5) определения количества договоров, заключенных на указанную услугу, по которым предоставлена скидка;
- 6) изменения названия фирмы, которая заключает договора и оказывает соответствующие услуги. Разработать статический метод, позволяющий изменять название фирмы.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 9

В программе разработать класс «Лекарство» со свойствами: код (целое число), название (строка), дата выпуска (структура «Дата»), стоимость (вещественное число), отдел (перечисление), наличие рецепта (логический тип), описание (строка), название аптеки (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о лекарствах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом лекарстве. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о лекарстве (по коду; по названию; по коду и названию);
- 3) отображения информации о лекарствах в виде таблицы;
- 4) поиска информации о лекарствах указанного года выпуска;
- 5) определения количества лекарств из указанного отдела и отпускающихся по рецепту;
- 6) изменения названия аптеки, в которой продаются лекарства. Разработать статический метод, позволяющий изменять название аптеки.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 10

В программе разработать класс «Фармацевт» со свойствами: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), адрес (строка), телефон (строка), пол (перечисление), дата рождения (структура «Дата»), оклад (вещественное число), наличие медицинского образования (логический тип), название аптеки (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о фармацевтах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом фармацевте. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о фармацевте (по табельному номеру; по Фамилии и Имени; по табельному номеру и Фамилии);
- 3) отображения информации о фармацевтах в виде таблицы;
- 4) поиска информации о фармацевтах указанного года рождения;
- 5) определения количества фармацевтов мужского пола с медицинским образованием;
- 6) изменения названия аптеки, в которой работают фармацевты. Разработать статический метод, позволяющий изменять название аптеки.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 11

В программе разработать класс «Преподаватель» со свойствами: табельный номер (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), адрес (строка), телефон (строка), преподаваемый предмет (строка), стаж работы (целое число), наличие ученой степени (логический тип), название университета (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о преподавателях хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом преподавателе. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о преподавателе (по табельному номеру; по Фамилии; по табельному номеру и Фамилии);
- 3) отображения информации о преподавателях в виде таблицы;
- 4) поиска информации о преподавателях указанного года рождения;
- 5) определения количества преподавателей мужского пола с ученой степенью;
- 6) изменения названия университета, в котором работают преподаватели.

Разработать статический метод, позволяющий изменять название университета.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 12

В программе разработать класс «Студент» со свойствами: номер зачетки (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), специальность (строка), наличие троек (логический тип), название университета (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о студентах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом студенте. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о студенте (по номеру зачетки; по Фамилии и Имени; по номеру зачетки и Фамилии);

- 3) отображения информации о студентах в виде таблицы;
- 4) поиска информации о студентах указанного года рождения;
- 5) определения количества студентов женского пола, обучающихся без строек;
- 6) изменения названия университета, в котором учатся студенты. Разработать статический метод, позволяющий изменять название университета.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 13

В программе разработать класс «Школа» со свойствами: код (целое число), название (строка), количество учеников (целое число), адрес (строка), телефон (строка), директор (структура «Директор»), тип (перечисление), государственная (логический тип), город (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о школах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новой школе. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о школе (по табельному коду; по названию; по коду и названию);
- 3) отображения информации о школах в виде таблицы;
- 4) поиска информации о школах с указанным директором (по фамилии);
- 5) определения количества государственных школ указанного типа;
- 6) изменения названия города, в котором расположены школы. Разработать статический метод, позволяющий изменять название города.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 14

В программе разработать класс «Университет» со свойствами: код (целое число), название (строка), город (перечисление), дата основания (структура «Дата»), ФИО ректора (строка), адрес (строка), телефон (строка), государственный (логический тип), страна (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию об университетах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом университете. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации об университете (по табельному коду; по названию; по коду и названию);
- 3) отображения информации об университетах в виде таблицы;
- 4) поиска информации об университетах, основанных в указанном году;
- 5) определения количества государственных университетов, расположенных в указанном городе;
- 6) изменения названия страны, в которой расположены университеты. Разработать статический метод, позволяющий изменять название страны.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 15

В программе разработать класс «Микроконтроллер» со свойствами: артикул (целое число), название (строка), фирма-производитель (структура «Фирма-производитель»), разрядность (целое число), тактовая частота (вещественное число), архитектура (перечисление), стоимость (вещественное число), уцененный товар (логический тип), название магазина (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о микроконтроллерах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом микроконтроллере. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о микроконтроллере (по артикулу; по названию; по артикулу и названию);
- 3) отображения информации о микроконтроллерах в виде таблицы;
- 4) поиска информации о микроконтроллерах указанной фирмы (по названию);
- 5) определения количества уцененных микроконтроллеров указанной архитектуры;
- 6) изменения названия магазина, в котором продаются микроконтроллеры.

Разработать статический метод, позволяющий изменять название магазина.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 16

В программе разработать класс «Ученик» со свойствами: код (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), класс (строка), наличие троек в четверти (логический тип), название школы (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию об учениках хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом ученике. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации об ученике (по коду; по Фамилии; по коду и Фамилии);
- 3) отображения информации об учениках в виде таблицы;
- 4) поиска информации об учениках указанного года рождения;
- 5) определения количества учеников мужского пола, закончивших четверть без троек;
- 6) изменения названия школы, в которой учатся ученики. Разработать статический метод, позволяющий изменять название школы.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 17

В программе разработать класс «Процессор» со свойствами: артикул (целое число), название (строка), фирма-производитель (структура «Фирма-производитель»), разрядность (целое число), тактовая частота (вещественное число), архитектура (перечисление), стоимость (вещественное число), уцененный товар (логический тип), название магазина (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о процессорах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом процессоре. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о процессоре (по артикулу; по названию; по артикулу и названию);
- 3) отображения информации о процессорах в виде таблицы;
- 4) поиска информации о процессорах указанной фирмы (по названию);
- 5) определения количества уцененных процессоров указанной архитектуры;
- 6) изменения названия магазина, в котором продаются процессоры. Разработать статический метод, позволяющий изменять название магазина.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 18

В программе разработать класс «Студент» со свойствами: номер зачетки (целое число), Фамилия (строка), Имя (строка), Отчество (строка), пол (перечисление), дата рождения (структура «Дата»), телефон (строка), адрес (строка), специальность (строка), наличие троек (логический тип), название университета (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о студентах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом студенте. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о студенте (по номеру зачетки; по Фамилии; по номеру зачетки и Фамилии);
- 3) отображения информации о студентах в виде таблицы;
- 4) поиска информации о студентах указанного года рождения;
- 5) определения количества студентов мужского пола, обучающихся без строек;
- 6) изменения названия университета, в котором учатся студенты. Разработать статический метод, позволяющий изменять название университета.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 19

В программе разработать класс «Лекарство» со свойствами: код (целое число), название (строка), дата выпуска (структура «Дата»), стоимость (вещественное число), отдел (перечисление), наличие рецепта (логический тип), описание (строка), название аптеки (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию о лекарствах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

- 1) добавления информации о новом лекарстве. Для добавления информации разработать конструкторы без параметров и с параметрами;
- 2) удаления информации о лекарстве (по коду; по названию и отделу; по коду и названию);
- 3) отображения информации о лекарствах в виде таблицы;
- 4) поиска информации о лекарствах указанного года выпуска;

5) определения количества лекарств из указанного отдела и отпускающихся без рецепта;

6) изменения названия аптеки, в которой продаются лекарства. Разработать статический метод, позволяющий изменять название аптеки.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Вариант № 20

В программе разработать класс «Университет» со свойствами: код (целое число), название (строка), город (перечисление), дата основания (структура «Дата»), ФИО ректора (строка), адрес (строка), телефон (строка), государственный (логический тип), страна (статическое поле). Все свойства класса являются закрытыми.

Вводимую информацию об университетах хранить в массиве объектов (статическом или динамическом; не использовать STL), объявленном в главной функции main.

Разработать подпрограммы для:

1) добавления информации о новом университете. Для добавления информации разработать конструкторы без параметров и с параметрами;

2) удаления информации об университете (по табельному коду; по названию и городу; по коду и названию);

3) отображения информации об университетах в виде таблицы;

4) поиска информации об университетах, основанных в указанном году;

5) определения количества государственных университетов, расположенных в указанном городе;

6) изменения названия страны, в которой расположены университеты. Разработать статический метод, позволяющий изменять название страны.

Разработать меню для вызова подпрограмм. Программа должна работать до тех пор, пока пользователь не решит из неё выйти. При вводе некорректных данных выдавать сообщение об ошибке.

Лабораторная работа № 3

Работа со строками (класс string)

Вариант №1

При вводе команды «One» программа должна выполнять задание 1, при вводе команды «Two» – задание 2, при вводе команды «Three» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество знаков препинания в тексте.
2. Задать с клавиатуры текст. Удалить все гласные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать текст и два слова (Слово1 и Слово2). Заменить в тексте Слово1 на Слово2, а Слово2 на Слово1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать char-массивы, а использовать класс string.

Вариант №2

При вводе команды «Z1» программа должна выполнять задание 1, при вводе команды «Z2» – задание 2, при вводе команды «Z3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество согласных букв в тексте.
2. Задать с клавиатуры текст. Удалить все знаки препинания в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед запятой не может быть пробела; после запятой должен стоять один пробел; не могут стоять подряд две запятые. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать char-массивы, а использовать класс string.

Вариант №3

При вводе команды «Task1» программа должна выполнять задание 1, при вводе команды «Task2» – задание 2, при вводе команды «Task3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество цифр в тексте.
2. Задать с клавиатуры текст. Удалить все согласные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед дефисом и после дефиса должны стоять по одному пробелу; не могут стоять подряд два дефиса. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать char-массивы, а использовать класс string.

Вариант №4

При вводе команды «T1» программа должна выполнять задание 1, при вводе команды «T2» – задание 2, при вводе команды «T3» – задание 3. При вводе неверной

команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте. Посчитать сколько раз встречается каждая буква в тексте.
2. Задать с клавиатуры текст. Удалить все цифры в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст и две буквы (Буква1 и Буква2). Заменить в тексте Букву1 на Букву2, а Букву2 на Букву1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №5

При вводе команды «One» программа должна выполнять задание 1, при вводе команды «Two» – задание 2, при вводе команды «Three» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте, начинающихся с согласной буквы.
2. Задать с клавиатуры текст. Удалить все запятые в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать текст и два слова (Слово1 и Слово2). Заменить в тексте Слово1 на Слово2, а Слово2 на Слово1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №6

При вводе команды «Z1» программа должна выполнять задание 1, при вводе команды «Z2» – задание 2, при вводе команды «Z3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество согласных букв в тексте.
2. Задать с клавиатуры текст. Удалить все заглавные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед дефисом и после дефиса должны стоять по одному пробелу; не могут стоять подряд два дефиса. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №7

При вводе команды «Task1» программа должна выполнять задание 1, при вводе команды «Task2» – задание 2, при вводе команды «Task3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество цифр в тексте.
2. Задать с клавиатуры текст. Удалить все строчные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: в начале строки не должно быть знаков препинания; в конце строки должна стоять точка; после каждой точки в тексте должен стоять пробел, за которым следует заглавная буква. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №8

При вводе команды «T1» программа должна выполнять задание 1, при вводе команды «T2» – задание 2, при вводе команды «T3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте, начинающихся на указанную пользователем букву.

2. Задать с клавиатуры текст. Удалить все точки в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать текст и два слова (Слово1 и Слово2). Заменить в тексте Слово1 на Слово2, а Слово2 на Слово1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №9

При вводе команды «One» программа должна выполнять задание 1, при вводе команды «Two» – задание 2, при вводе команды «Three» – задание . При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество цифр в тексте.

2. Задать с клавиатуры текст. Удалить все пробелы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед дефисом и после дефиса должны стоять по одному пробелу; не могут стоять подряд два дефиса. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №10

При вводе команды «Z1» программа должна выполнять задание 1, при вводе команды «Z2» – задание 2, при вводе команды «Z3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте, начинающихся с гласной буквы.

2. Задать с клавиатуры текст. Удалить указанную букву в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед запятой не может быть пробела; после запятой должен стоять один пробел; не могут стоять подряд две запятые. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №11

При вводе команды «Task1» программа должна выполнять задание 1, при вводе команды «Task2» – задание 2, при вводе команды «Task3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество букв в тексте.
2. Задать с клавиатуры текст. Удалить указанную цифру в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: в начале строки не должно быть знаков препинания; в конце строки должна стоять точка; после каждой точки в тексте должен стоять пробел, за которым следует заглавная буква. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №12

При вводе команды «T1» программа должна выполнять задание 1, при вводе команды «T2» – задание 2, при вводе команды «T3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте с тремя гласными буквами.
2. Задать с клавиатуры текст. Удалить указанный знак препинания в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать текст и два слова (Слово1 и Слово2). Заменить в тексте Слово1 на Слово2. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №13

При вводе команды «One» программа должна выполнять задание 1, при вводе команды «Two» – задание 2, при вводе команды «Three» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте с тремя согласными буквами.
2. Задать с клавиатуры текст. Удалить все цифры в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед запятой не может быть пробела; после запятой должен стоять один пробел; не могут стоять подряд две запятые; предложение не может начинаться с запятой. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №14

При вводе команды «Z1» программа должна выполнять задание 1, при вводе команды «Z2» – задание 2, при вводе команды «Z3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов, в которых встречается указанная пользователем буква.

2. Задать с клавиатуры текст. Удалить все запятые в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать текст и два слова (Слово1 и Слово2). Заменить в тексте Слово1 на Слово2, а Слово2 на Слово1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №15

При вводе команды «One» программа должна выполнять задание 1, при вводе команды «Two» – задание 2, при вводе команды «Three» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте, начинающихся с заглавной буквы.

2. Задать с клавиатуры текст. Удалить все гласные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать с клавиатуры текст. Отредактировать текст по следующим правилам: перед дефисом и после дефиса должны стоять по одному пробелу; не могут стоять подряд два дефиса; предложение не может начинаться с дефиса. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №16

При вводе команды «Task1» программа должна выполнять задание 1, при вводе команды «Task2» – задание 2, при вводе команды «Task3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте, начинающихся со строчной буквы.

2. Задать с клавиатуры текст. Удалить все согласные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать текст и два слова (Слово1 и Слово2). Заменить в тексте Слово1 на Слово2, а Слово2 на Слово1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №17

При вводе команды «Task1» программа должна выполнять задание 1, при вводе команды «Task2» – задание 2, при вводе команды «Task3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество слов в тексте из трёх букв.

2. Задать с клавиатуры текст. Удалить все строчные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.

3. Задать текст и два словосочетания (Словосочетание1 и Словосочетание2). Заменить в тексте Словосочетание1 на Словосочетание2. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №18

При вводе команды «One» программа должна выполнять задание 1, при вводе команды «Two» – задание 2, при вводе команды «Three» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать количество букв в тексте. Посчитать сколько раз встречается каждая буква в тексте.
2. Задать с клавиатуры текст. Удалить все заглавные буквы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст и две буквы (Буква1 и Буква2). Заменить в тексте Букву1 на Букву2, а Букву2 на Букву1. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №19

При вводе команды «Z1» программа должна выполнять задание 1, при вводе команды «Z2» – задание 2, при вводе команды «Z3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать сколько раз встречается каждый символ в тексте.
2. Задать с клавиатуры текст. Удалить все пробелы в тексте. Вывести содержимое строкой переменной с отредактированным текстом на экран.
3. Задать с клавиатуры текст. Отредактировать текст по следующему правилу: перед дефисом и после дефиса обязательно должны стоять по одному пробелу. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Вариант №20

При вводе команды «Task1» программа должна выполнять задание 1, при вводе команды «Task2» – задание 2, при вводе команды «Task3» – задание 3. При вводе неверной команды выдать сообщение об ошибке. Программа должна обеспечить возможность выполнять задания до тех пор, пока пользователь не решит из неё выйти. Задания:

1. Задать с клавиатуры текст. Посчитать сколько раз встречается заданное пользователем слово.
2. Задать с клавиатуры текст. Удалить все запятые в тексте. Вывести содержимое строковой переменной с отредактированным текстом на экран.
3. Задать текст и два словосочетания (Словосочетание1 и Словосочетание2). Заменить в тексте Словосочетание1 на Словосочетание2. Вывести содержимое строковой переменной с отредактированным текстом на экран.

При разработке программы не использовать `char`-массивы, а использовать класс `string`.

Лабораторная работа № 4

Перегрузка операций и преобразование типов

Вариант 1

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): сложение, вычитание, инкремент постфиксный и инкремент префиксный (увеличение даты на один день), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 2

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): сложение, деление, инкремент постфиксный и инкремент префиксный (увеличение значений всех элементов на 1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 3

Разработать класс для работы с прямоугольной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции сложения и вычитания, декремент постфиксный и декремент префиксный (уменьшение значений всех элементов на 0.01), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 4

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): сложение, вычитание, декремент постфиксный и декремент префиксный (уменьшение даты на один день), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 5

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): вычитание, умножение, декремент постфиксный и декремент префиксный (уменьшение значений всех элементов на 1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 6

Разработать класс для работы с прямоугольной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): сложение, вычитание, инкремент постфиксный и инкремент префиксный (увеличение значений всех элементов на 10), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 7

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): сложение, деление, инкремент постфиксный и инкремент префиксный (увеличение значений всех элементов на 0.1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 8

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции сложения и вычитания, инкремент постфиксный и инкремент префиксный (увеличение на одну минуту), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 9

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции вычитания и умножения, инкремент постфиксный и инкремент префиксный (увеличение значений всех элементов на 0.1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 10

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): сложение, вычитание, инкремент постфиксный и инкремент префиксный (увеличение времени на одну секунду), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 11

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции вычитания и умножения, инкремент постфиксный и инкремент префиксный (увеличение значений всех элементов на 1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 12

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): сложение, операции отношения, декремент постфиксный и декремент префиксный (уменьшение на одну секунду), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 13

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции сложения и деления, декремент постфиксный и декремент префиксный (уменьшение значений всех элементов на 0.1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 14

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): сложение, вычитание, инкремент постфиксный и инкремент префиксный (увеличение даты на один год), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 15

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): вычитание, умножение, декремент постфиксный и декремент префиксный (уменьшение значений всех элементов на 0.1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 16

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): сложение, комбинированное сложение, инкремент постфиксный и инкремент префиксный (увеличение на час), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 17

Разработать класс для работы с прямоугольной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции сложения и вычитания, декремент постфиксный и декремент префиксный (уменьшение значений всех элементов на 0.01), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 18

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): комбинированные операции сложения и деления, декремент постфиксный и декремент префиксный (уменьшение значений всех элементов на 1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 19

Разработать класс «Дата-Время».

Реализовать набор операций для работы с классом (перегруженные операции): вычитание, операции отношения, декремент постфиксный и декремент префиксный (уменьшение на одну секунду), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Вариант 20

Разработать класс для работы с квадратной матрицей.

Реализовать набор операций для работы с классом (перегруженные операции): сложение, деление, инкремент постфиксный и инкремент префиксный (увеличение значений всех элементов на 0.1), ввод информации, вывод информации.

В программе реализовать преобразование типов (от основного к пользовательскому, от пользовательского к основному, от пользовательского к пользовательскому).

Лабораторная работа № 5

Наследование

В программе разработать базовый класс и производные классы в соответствии с вариантом задания. Поля базового класса определить самостоятельно (2-3 поля). Производные классы обязательно должны содержать новые поля (1-3 поля, производные классы должны содержать разное количество новых полей). Для базового и производных классов обязательно разработать виртуальную функцию.

При разработке классов грамотно использовать основные принципы объектно-ориентированного программирования (инкапсуляция, наследование, полиморфизм).

Создать массив объектов базового класса для заполнения его объектами производных классов (для хранения данных (объекты производных классов) в программе создаете ОДИН массив типа базового класса и с этим массивом работаете – добавляете в него данные (объекты производных классов) и удаляете из него данные (объекты производных классов); массивы объектов производных классов создавать не надо).

Функционал программы: добавление данных (объект производного класса) в массив; удаление данных (объект производного класса) из массива; отображение информации, содержащейся в массиве, в виде таблиц(ы).

Разработать удобный графический пользовательский интерфейс.

Варианты заданий:

1. Базовый класс «Работник фирмы», производные классы «Менеджер» и «Бухгалтер».
2. Базовый класс «Человек», производные классы «Преподаватель» и «Студент».
3. Базовый класс «Музыкальный инструмент», производные классы «Ударный» и «Струнный».
4. Базовый класс «Транспортное средство», производные классы «Самолет» и «Автомобиль».
5. Базовый класс «Птица», производные классы «Дикая» и «Домашняя».
6. Базовый класс «Фигура», производные классы «Круг» и «Прямоугольник».
7. Базовый класс «Товар», производные классы «Продовольственный» и «Непродовольственный».
8. Базовый класс «Учащийся», производные классы «Школьник» и «Студент».
9. Базовый класс «Транспортное средство», производные классы «Сухопутное» и «Летающее».
10. Базовый класс «Человек», производные классы «Продавец» и «Покупатель».

11. Базовый класс «Технический работник», производные классы «Слесарь» и «Уборщик».

12. Базовый класс «Дикое животное», производные классы «Травоядное» и «Плотоядное».

13. Базовый класс «Работник», производные классы «Кассир» и «Охранник».

14. Базовый класс «Учебное заведение» производные классы «Школа» и «Университет».

15. Базовый класс «Лекарственное средство», производные классы «Витамины» и «Антибиотик».

16. Базовый класс «Фигура», производные классы «Трапеция» и «Квадрат».

17. Базовый класс «Животное», производные классы «Дикое» и «Домашнее».

18. Базовый класс «Транспортное средство», производные классы «Наземное» и «Водное».

19. Базовый класс «Человек», производные классы «Учитель» и «Ученик».

20. Базовый класс «Одежда», производные классы «Летняя» и «Зимняя».

Лабораторная работа № 6

Работа со связным списком

Для хранения данных организовать связный список в соответствии с вариантом задания. Функционал программы: добавление данных (элемента) в список; удаление данных (элемента) из списка; отображение информации, содержащейся в списке, в виде таблицы.

Разработать удобный графический пользовательский интерфейс.

Варианты заданий:

1. Класс «Сотрудник», двусвязный циклический список.
2. Класс «Учитель», двусвязный линейный список.
3. Класс «Ученик», односвязный циклический список.
4. Класс «Книга», двусвязный циклический список.
5. Класс «Журнал», двусвязный циклический список.
6. Класс «Товар», двусвязный линейный список.
7. Класс «Продавец», односвязный циклический список.
8. Класс «Договор», двусвязный линейный список.
9. Класс «Лекарство», двусвязный циклический список.
10. Класс «Фармацевт», двусвязный линейный список.
11. Класс «Преподаватель», односвязный циклический список.
12. Класс «Студент», двусвязный циклический список.
13. Класс «Школа», двусвязный циклический список.
14. Класс «Университет», двусвязный линейный список.
15. Класс «Микроконтроллер», односвязный циклический список.
16. Класс «Ученик», двусвязный циклический список.
17. Класс «Процессор», двусвязный циклический список.
18. Класс «Студент», двусвязный линейный список.
19. Класс «Лекарство», односвязный циклический список.
20. Класс «Университет», двусвязный циклический список.

Лабораторная работа № 7

Работа со стандартной библиотекой шаблонов STL

В программе разработать класс в соответствии с вариантом задания.

Программу представить в двух вариантах:

- 1) с использованием контейнера «Вектор» (vector) для хранения информации об объектах;
- 2) с использованием контейнера «Список» (list) для хранения информации об объектах.

Функционал программы: добавление данных (объекта) в контейнер; удаление данных (объекта) из контейнера; отображение информации, содержащейся в контейнере, в виде таблицы.

Разработать удобный графический пользовательский интерфейс.

Варианты заданий:

1. Класс «Сотрудник».
2. Класс «Учитель».
3. Класс «Ученик».
4. Класс «Книга».
5. Класс «Журнал».
6. Класс «Товар».
7. Класс «Продавец».
8. Класс «Договор».
9. Класс «Лекарство».
10. Класс «Фармацевт».
11. Класс «Преподаватель».
12. Класс «Студент».
13. Класс «Школа».
14. Класс «Университет».
15. Класс «Микроконтроллер».
16. Класс «Ученик».
17. Класс «Процессор».
18. Класс «Студент».
19. Класс «Лекарство».
20. Класс «Университет».